

Architecture Specification Document

Parameterizable Output-Stationary Systolic Array AI Accelerator

SSCS Chipathon 2026 | Track A | Slot A

PDK: GF180MCU (gf180mcu_fd_sc_mcu7t5v0, 7-track, 5V)

Target: Slot A | Target Frequency: 25 MHz

Team Name: Maxilerator

Members: Irene Raphael, Amal Kunnath Anil Narayana, Muhammed Rabin K C, Muhammad Faqih Ilmi, Akhil S Nair

University: TU Munich, NTUST, IIT Delhi

Version: v2.0

Section 1 — Overview and Objectives

1.1 Design Description

This document describes the architecture of a parameterizable output-stationary systolic array AI accelerator designed for submission to the SSCS Chipathon 2026 Track A. The design implements streaming tiled matrix-matrix multiplication targeting edge inference workloads from TinyML to large CNN and transformer models. Activations and weights are consumed directly as they stream in over two independent AXI4-Stream channels — there is no on-chip buffer memory and no dedicated FSM controller. Each module in the design self-generates its own phase transition from its own inputs and hands that signal directly to the next stage, removing the need for a centralized coordinator.

The accelerator supports matrices larger than its native 8×8 compute array through a tiling strategy: multiple passes stream through back to back with no gap between them, and the array only drains and begins producing output once the host has signalled the true final word of the entire stream, not after each individual pass.

1.2 Design Goals

Functional Goals

- Implement a fully functional 8×8 output-stationary systolic array
- Support signed matrix multiplication with 21-bit accumulation (INT8 operands by default, DATA_WIDTH configurable)
- Support streaming tiled computation for matrices larger than 8×8, with no on-chip buffer memory
- Self-generate all phase transitions (drain, output start, reset) with no dedicated controller
- Target operating frequency: 25 MHz on GF180MCU

Reusability Goals

- Publish `accelerator_core` as a standalone reusable IP with a native three-channel AXI4-Stream interface
- Keep `systolic_array` and `mac_unit` as maximally reusable arithmetic primitives, independent of any interface concern
- Maintain a clean separation between compute/control logic (`accelerator_core` and below) and physical pad integration (`host_interface/chip_core`)
- Support a future wider-bus AXI4-Stream SoC wrapper without any change to `accelerator_core` itself

Verification Goals

- Verify every module using cocotb Python-based simulation, with a NumPy software golden model
- Verify the full pad-level physical interface via a dedicated chip-level cocotb testbench, driving the actual GF180MCU I/O pad models
- Demonstrate correct end-to-end matrix multiplication through the real physical pad interface prior to tapeout

1.3 Key Design Decisions

- Output-stationary dataflow: accumulators persist across all streaming passes, cleared only after the final result of a full computation has been transferred out
- No on-chip buffer memory: feeder consumes activation and weight bytes directly as they arrive, with no SRAM or other storage stage between the host and the systolic array

- No controller module: every phase transition is self-generated by the module that owns it, and wired directly to the next stage
- feeder self-generates drain_done from its own observation of the true final activation/weight word (a_last/b_last), independent of any external coordination
- output_processor self-generates output_done the instant its own final result transfers, and that single pulse is wired externally to reset both feeder and the MAC array
- Area-optimized radix-2 Booth multiplier inside mac_unit, using a deferred-correction-bit technique to avoid per-partial-product negation adders
- Simple valid/ready handshake at every accelerator_core channel boundary, matching standard AXI4-Stream semantics
- host_interface handles the physical pin protocol for Chipathon testing only, sharing a single physical data bus across all three logical channels to fit Slot A's pin budget

Section 2 — System Architecture

2.1 Output-Stationary Dataflow

The design uses output-stationary dataflow where each MAC unit holds its accumulator stationary across all streaming passes. For an $N \times N$ matrix multiply $C = A \times B$:

- A values enter the LEFT edge of the array and flow RIGHT through pipeline registers, one hop per cycle
- B values enter the TOP edge of the array and flow DOWN through pipeline registers, one hop per cycle
- Each $MAC[i][j]$ accumulates $A[i][k] \times B[k][j]$ for all inner steps k across all passes
- Accumulators are NEVER reset between passes
- Accumulators reset ONLY after the true final result of the entire computation has been transferred out
- `mac_unit's accum_out` is combinational from the accumulator register: valid one cycle after the corresponding valid pulse

2.2 Streaming and Tiling Strategy

For matrices larger than `ARRAY_SIZE` $\times$ `ARRAY_SIZE`, the computation is split into passes of one `ARRAY_SIZE`-deep block each. Each pass streams one column of A and one row of B per cycle-step, skewed diagonally by feeder so that every MAC receives its correct operand pair on the correct cycle. There is no host-visible per-pass boundary signal — the host simply streams all passes back to back, and asserts `a_last/b_last` only on the true final activation/weight byte of the entire computation.

Example — 32 $\times$ 32 matrix multiply using an 8 $\times$ 8 array (4 passes required):

```
C = A x B where A, B, C are 32x32 matrices
Number of passes = 32 / 8 = 4
```

```
Computing output tile C[0:7][0:7]:
```

```
Pass 1: A[0:7][0:7]   x B[0:7][0:7]   -> partial sum
Pass 2: A[0:7][8:15]  x B[8:15][0:7]  -> adds to accumulator
Pass 3: A[0:7][16:23] x B[16:23][0:7] -> adds to accumulator
Pass 4: A[0:7][24:31] x B[24:31][0:7] -> final sum
```

`a_last/b_last` asserted only on the TRUE final byte of pass 4 -- feeder self-detects this and drains automatically; no separate signal marks the boundary between passes 1-2-3-4.

After the final pass, feeder self-drains and `output_processor` serializes the 64 results, then self-generates the clear pulse that resets the whole chip for the next computation — no external coordination is required anywhere in this sequence.

2.3 Usage Scenarios

Scenario 1 — Chipathon Physical Chip

```
Laptop (Python) -> USB/FTDI -> PCB -> physical pins
-> chip_core (pad index mapping)
-> host_interface (shared-bus protocol adapter)
-> accelerator_core
```

`host_interface` translates the shared, pin-budget-constrained physical protocol into `accelerator_core's` native three-channel interface. It is used only for Chipathon testing and validation, and is not part of the reusable IP boundary.

Scenario 2 — IP Reuse in an SoC

accelerator_core's native interface is already AXI4-Stream-shaped (independent data/valid/ready/last per channel, one channel per operand plus one for results) and can be instantiated directly by any SoC integration with sufficient pin/bus width, with no adapter needed at all. A dedicated wide-bus AXI4-Stream wrapper is a planned future addition for environments that specifically require full AXI4-Stream protocol compliance (see Section 8); accelerator_core itself never needs to change to support it.

Section 3 — Module Hierarchy

3.1 Full Hierarchy

```
chip_top                                <- fixed Chipathon padring template
|-- chip_core                          <- Slot A pin mapping (physical pads only)
  |-- host_interface                   <- protocol adapter: shared pad bus <-> AXI4-Stream
    |-- accelerator_core               <- reusable IP boundary
      |-- feeder                       <- dual-lane input, skew buffer, self-generated drain
      |-- systolic_array               <- 8x8 MAC grid
        |-- mac_unit [x64]             <- signed integer MAC primitive
          |-- booth_multiplier
          |-- booth_encoder
          |-- partial_product_generator
      |-- output_processor              <- result serialization, quantization, self-generated
done
```

chip_top is the fixed Chipathon padframe template shared across all slots and is not modified for this project. chip_core is the only slot-specific integration layer: it knows nothing about the accelerator's protocol, and does nothing beyond mapping specific physical pad array indices to host_interface's named ports and setting per-pad electrical configuration (input threshold, slew rate, pull resistors, direction). accelerator_core exposes a native three-channel AXI4-Stream interface and has no knowledge of physical pins at all; host_interface exists purely to fit that interface onto Slot A's pin budget by sharing one physical data bus across all three logical channels.

3.2 Reusability Levels

Level	Modules	Description
1 — Most Reusable	mac_unit, systolic_array, booth_multiplier and submodules	Pure arithmetic, no memory, control, or interface dependency
2 — Core IP	accelerator_core, feeder, output_processor	Full accelerator with a native AXI4-Stream handshake interface, reusable in any SoC
3 — System Specific	host_interface, chip_core	Chipathon Slot A tapeout only
4 — Fixed Template	chip_top	Shared Chipathon padframe template, not modified

Section 4 — Module Specifications

Each module specification includes: purpose, parameters, port declaration with descriptions, internal operation, and handshaking with neighboring modules.

4.1 chip_top

Purpose

Fixed Chipathon padframe template, shared across all slots and all teams. Instantiates the physical GF180MCU pad cells (input-only, bidirectional, corner, power) according to the parameters supplied by the active slot definition, and instantiates chip_core with the resulting pad-level signal arrays. This module is not modified for this project — all project-specific integration happens one level down, in chip_core.

4.2 chip_core

Purpose

The Slot A pin-mapping layer. Knows only about raw, index-based pad arrays as exposed by chip_top (bidir_in/out/oe/cs/sl/ie/pu/pd, input_in/pu/pd) — no protocol logic lives here, only wire connections from specific array indices to host_interface's named ports, plus per-pin electrical configuration.

Parameters

Parameter	Value	Description
NUM_INPUT_PADS	5	Input-only pad count (a_valid_i, a_last_i, b_valid_i, b_last_i, out_ready_i)
NUM_BIDIR_PADS	12	Bidirectional pad count (data[7:0] + a_ready_o, b_ready_o, out_valid_o, out_last_o)
NUM_ANALOG_PADS	1	Reserved workaround pad only, not functionally used (see Section 6)

Pin Map

```
input_PAD[0]  a_valid_i      input_PAD[3]  b_last_i
input_PAD[1]  a_last_i       input_PAD[4]  out_ready_i
input_PAD[2]  b_valid_i

bidir_PAD[0:7] data[7:0]      bidir_PAD[10] out_valid_o
bidir_PAD[8]  a_ready_o       bidir_PAD[11] out_last_o
bidir_PAD[9]  b_ready_o
```

Internal Operation

- bidir_oe[7:0] (the shared data bus) is driven directly by host_interface's own data_oe output, so direction switches automatically the instant host_interface's internal out_valid_o rises
- bidir_oe[11:8] (a_ready_o, b_ready_o, out_valid_o, out_last_o) are permanently tied to 1 — these four signals are always chip-driven
- bidir_ie is derived as the complement of bidir_oe on every pin — a pin only listens while it is not driving
- bidir_cs/bidir_sl set to CMOS input threshold and fast slew for every pin; bidir_pu/bidir_pd and input_pu/input_pd are all tied to 0 (no pull resistors)

4.3 host_interface

Purpose

Protocol adapter between accelerator_core's native three-channel AXI4-Stream interface and Slot A's narrow, shared physical pin budget. Not part of the reusable IP boundary — a different physical integration target would only need a different adapter at this same boundary, with accelerator_core itself unchanged.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Propagated to accelerator_core
DATA_WIDTH	8	Propagated to accelerator_core
ACCUM_WIDTH	21	Propagated to accelerator_core
SHIFT_BITS	0	Propagated to accelerator_core
SAT_MAX / SAT_MIN	derived from DATA_WIDTH	Propagated to accelerator_core

Port Declaration

```
module host_interface #(
    parameter ARRAY_SIZE = 8,
    parameter DATA_WIDTH = 8,
    parameter ACCUM_WIDTH = 21,
    parameter SHIFT_BITS = 0,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MAX = (1 <<< (DATA_WIDTH-1)) - 1,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MIN = -(1 <<< (DATA_WIDTH-1))
) (
    input logic clk,
    input logic rst_n,

    input logic [DATA_WIDTH-1:0] data_i,
    output logic [DATA_WIDTH-1:0] data_o,
    output logic data_oe,

    input logic a_valid_i,
    output logic a_ready_o,
    input logic a_last_i,

    input logic b_valid_i,
    output logic b_ready_o,
    input logic b_last_i,

    output logic out_valid_o,
    input logic out_ready_i,
    output logic out_last_o
);
```

Internal Operation

- data_i is tapped directly to both accelerator_core.a_data and accelerator_core.b_data — safe because feeder only ever latches whichever lane has valid=1 asserted, and the host is required (by protocol) to never assert a_valid_i and b_valid_i in the same cycle
- data_o is wired directly from accelerator_core.out_data — there is only ever one possible source, so no selection logic is needed on the output side
- data_oe = out_valid_o (accelerator_core's own out_valid) — direction on the shared bus is decided entirely by the chip's own knowledge of its current phase, with no separate state machine
- Every other control line (a_valid/a_ready/a_last, b_valid/b_ready/b_last, out_valid/out_ready/out_last) has a single fixed owner for its entire lifetime — host-driven or chip-driven, never both — so none of them need direction control

4.4 accelerator_core

Purpose

The reusable IP boundary. Purely structural — instantiates feeder, systolic_array, and output_processor and wires them together. Contains no logic of its own; every actual behavior lives in the three submodules it connects.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Systolic array dimension (any power of 2)
DATA_WIDTH	8	Activation/weight/result bit width
ACCUM_WIDTH	21	Accumulator width; 21-bit supports $K \leq 32$ without overflow
SHIFT_BITS	0	Right-shift applied before saturation (0 = pure clip)
SAT_MAX	$2^{(\text{DATA_WIDTH}-1)}-1$	Upper saturation bound, overridable for tighter quantization calibration
SAT_MIN	$-2^{(\text{DATA_WIDTH}-1)}$	Lower saturation bound

Port Declaration

```
module accelerator_core #(
    parameter ARRAY_SIZE = 8,
    parameter DATA_WIDTH = 8,
    parameter ACCUM_WIDTH = 21,
    parameter SHIFT_BITS = 0,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MAX = (1 <<< (DATA_WIDTH-1)) - 1,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MIN = -(1 <<< (DATA_WIDTH-1))
) (
    input logic clk,
    input logic rst_n,

    input logic [DATA_WIDTH-1:0] a_data,
    input logic a_valid,
    output logic a_ready,
    input logic a_last,

    input logic [DATA_WIDTH-1:0] b_data,
    input logic b_valid,
    output logic b_ready,
    input logic b_last,

    output logic [DATA_WIDTH-1:0] out_data,
    output logic out_valid,
    input logic out_ready,
    output logic out_last
);
```

Internal Wiring

```
feeder.drain_done          -> output_processor.output_en
output_processor.output_done -> feeder.clear AND systolic_array.clear
feeder.a_out, feeder.b_out, feeder.valid -> systolic_array.a_in, b_in, valid
systolic_array.results -> output_processor.results
```

No output_done-equivalent signal is exposed at the accelerator_core boundary. out_last already gives any consumer everything needed to detect the final result byte, in exactly the same way a_last/b_last work on the input side — a consumer derives "transfer complete" itself via out_valid && out_ready && out_last, matching how accelerator_core derives the same fact internally.

4.5 feeder

Purpose

Receives activation (A) and weight (B) words on two independent AXI4-Stream channels, synchronizes them vector by vector (one column of A, one row of B per vector), skews them diagonally for the systolic array, and self-triggers a pipeline flush once the host signals the true final word on both channels. Has no internal buffer memory — every word is consumed and shifted directly into the skew buffer as it arrives.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Controls staging register count, skew depth, and drain counter width
DATA_WIDTH	8	Activation/weight word width

Port Declaration

```
module feeder #(
    parameter ARRAY_SIZE = 8,
    parameter DATA_WIDTH = 8
) (
    input  logic                clk,
    input  logic                rst_n,

    input  logic [DATA_WIDTH-1:0] a_data,
    input  logic                a_valid,
    output logic                a_ready,
    input  logic                a_last,

    input  logic [DATA_WIDTH-1:0] b_data,
    input  logic                b_valid,
    output logic                b_ready,
    input  logic                b_last,

    input  logic                clear,

    output logic [ARRAY_SIZE*DATA_WIDTH-1:0] a_out,
    output logic [ARRAY_SIZE*DATA_WIDTH-1:0] b_out,
    output logic                valid,

    output logic                drain_done
);
```

Internal Structure

- **Input Sync and Handshaking:** standard AXI4-Stream two-lane handshake, per-vector position counters, vector/stream completion latches, a_ready/b_ready generation. A vector completes (and a new wavefront is presented to the array) only once both lanes have delivered all ARRAY_SIZE words for that vector — A and B words may arrive in either order and independently stall.
- **Drain Sequencer:** self-triggered flush once drain_en (both lanes' true final word observed) latches. Continues shifting zeros through the skew buffer for 2x(ARRAY_SIZE-1) cycles — enough for the deepest row's final real value to finish propagating through both the skew buffer and the array itself — then latches drain_done one cycle after the last drain pulse, giving the array's own registered accumulators time to settle.
- **Staging Registers:** a_stage[]/b_stage[] capture each lane's current vector as it arrives, forced to zero during the drain sequence.

- Skew Buffer: row 0 is a direct wire from staging (zero delay); rows 1 through ARRAY_SIZE-1 each have i register stages, so row i is delayed by i pulses relative to row 0. This produces the diagonal wavefront pattern the systolic array requires.

Handshaking with Neighboring Modules

Neighbor	Signals	Description
Host / accelerator_core boundary	a_data, a_valid, a_last, b_data, b_valid, b_last (in); a_ready, b_ready (out)	Standard two-lane AXI4-Stream input handshake
output_processor	clear (in)	Single-cycle pulse; resets all staging, skew, and counter state
systolic_array	a_out, b_out, valid (out)	Skewed data and single-cycle valid pulses to the array's edges

4.6 systolic_array

Purpose

An ARRAY_SIZE x ARRAY_SIZE grid of mac_unit processing elements, wired in the standard systolic dataflow: a_in flows left to right (row-wise), b_in flows top to bottom (column-wise), each PE forwarding what it received to its eastern/southern neighbor one cycle later. Has no awareness of skewing itself — it only wires PEs together and lets each one forward its inputs one cycle later, which is what produces the diagonal propagation feeder's skew buffer is designed around.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Grid dimension (N x N MAC units)
DATA_WIDTH	8	Input operand width
ACCUM_WIDTH	21	Accumulator width

Port Declaration

```
module systolic_array #(
    parameter ARRAY_SIZE = 8,
    parameter DATA_WIDTH = 8,
    parameter ACCUM_WIDTH = 21
) (
    input logic clk,
    input logic rst_n,

    input logic [ARRAY_SIZE*DATA_WIDTH-1:0] a_in,
    input logic [ARRAY_SIZE*DATA_WIDTH-1:0] b_in,
    input logic valid,
    input logic clear,

    output logic [ARRAY_SIZE*ARRAY_SIZE*ACCUM_WIDTH-1:0] results
);
```

MAC Unit Grid Connectivity

```
a_in[i] -> PE(i,0).a_in      (left edge, row i)
b_in[j] -> PE(0,j).b_in      (top edge, column j)
PE(i,c).a_out -> PE(i,c+1).a_in  (A flows right)
PE(r,j).b_out -> PE(r+1,j).b_in  (B flows down)
PE(i,j).accum_out -> results[i][j] (packed into the flat results bus)
```

valid and clear fan out identically to every PE in the array. Output timing: mac_unit has 1-cycle latency, so accum_out is valid one cycle after a valid pulse. After feeder's drain phase completes (drain_done asserted), all $ARRAY_SIZE^2$ MAC accum_out values are guaranteed valid and stable.

4.7 mac_unit

Purpose

A single processing element (PE) of the systolic array. Each cycle it receives a_in from its western neighbor (or the array's left edge) and b_in from its northern neighbor (or the array's top edge), multiplies them via booth_multiplier, and accumulates the result into accum_out — output-stationary: accum_out holds the running partial sum across every valid pulse for the whole computation, only clearing between runs.

Parameters

Parameter	Default	Description
DATA_WIDTH	8	Input operand width
ACCUM_WIDTH	21	Accumulator register width

Port Declaration

```
module mac_unit #(
    parameter DATA_WIDTH = 8,
    parameter ACCUM_WIDTH = 21
) (
    input logic          clk,
    input logic          rst_n,

    input logic signed [DATA_WIDTH-1:0] a_in,
    input logic signed [DATA_WIDTH-1:0] b_in,
    input logic          valid,
    input logic          clear,

    output logic signed [DATA_WIDTH-1:0] a_out,
    output logic signed [DATA_WIDTH-1:0] b_out,
    output logic signed [ACCUM_WIDTH-1:0] accum_out
);
```

RTL Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        a_out <= 0; b_out <= 0; accum_out <= 0;
    end else if (clear) begin
        a_out <= 0; b_out <= 0; accum_out <= 0;
    end else if (valid) begin
        a_out <= a_in;
        b_out <= b_in;
        accum_out <= accum_out + booth_product(a_in, b_in);
    end
end
```

rst_n and clear are kept as separate branches, not combined into one condition, since the target Yosys flow does not support the combined form for this construct. a_out/b_out are reset on both rst_n and clear — a partial fix from earlier revisions, where only accum_out was cleared, left stale neighbor data able to corrupt the next computation's early accumulation.

4.8 booth_multiplier, booth_encoder, partial_product_generator

Purpose

A radix-2 Booth multiplier providing mac_unit's signed multiply, chosen over a plain $a*b$ expression for area efficiency confirmed on the target GF180MCU synthesis flow. booth_encoder decodes each 3-bit overlapping window of the multiplier operand into zero/sel2/negate control signals; partial_product_generator forms each partial product row using a deferred-correction-bit technique that avoids a full two's-complement negation adder per row, instead accumulating a single correction bit at the appropriate bit position. booth_multiplier combines the encoder and partial product generator outputs into the final signed product.

These are the most reusable modules in the design: pure combinational arithmetic, with no interface, control, or memory dependency of any kind.

4.9 output_processor

Purpose

Serializes the systolic array's $ARRAY_SIZE \times ARRAY_SIZE$ accumulator results into a byte-wide AXI4-Stream output, one result per cycle, row-major. Each result is optionally shifted (SHIFT_BITS) then saturated to [SAT_MIN, SAT_MAX] before truncation to DATA_WIDTH. Self-generates output_done — a single-cycle pulse that fires the exact cycle the true final result transfers — which both resets this module's own row/col counters and is wired externally to feeder's and the MAC array's clear inputs.

Parameters

Parameter	Default	Description
ARRAY_SIZE	8	Controls result count ($ARRAY_SIZE^2$) and row/col counter width
ACCUM_WIDTH	21	Input accumulator width before quantization
DATA_WIDTH	8	Output byte width
SHIFT_BITS	0	Right-shift applied before saturation
SAT_MAX / SAT_MIN	derived from DATA_WIDTH	Saturation bounds, overridable for quantization calibration

Port Declaration

```
module output_processor #(
    parameter ARRAY_SIZE = 8,
    parameter ACCUM_WIDTH = 21,
    parameter DATA_WIDTH = 8,
    parameter SHIFT_BITS = 0,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MAX = (1 <<< (DATA_WIDTH-1)) - 1,
    parameter signed [ACCUM_WIDTH-1:0] SAT_MIN = -(1 <<< (DATA_WIDTH-1))
) (
    input logic clk,
    input logic rst_n,

    input logic [ARRAY_SIZE*ARRAY_SIZE*ACCUM_WIDTH-1:0] results,

    input logic output_en,
    output logic output_done,

    input logic out_ready,
    output logic [DATA_WIDTH-1:0] out_data,
    output logic out_valid,
    output logic out_last
);
```

Internal Operation

- Result selection: results is unpacked into a genuine 2D array using constant-index generate assignments (free for synthesis), then selected in two stages — row first, then column within that row. Row is selected first specifically because row only changes once every ARRAY_SIZE cycles during serialization while column changes every cycle, so the wider first-stage mux's output stays stable for ARRAY_SIZE consecutive cycles instead of toggling every cycle, reducing switching activity for the same silicon area. This two-stage index-match structure was chosen over both a flat variable-index part-select (which synthesizes unpredictably) and a hierarchical mux4 reduction tree (functionally correct, but a less area-efficient synthesis outcome in practice on this flow).
- Quantization: the selected raw accumulator value is arithmetically right-shifted by SHIFT_BITS, then compared against SAT_MAX/SAT_MIN and clamped if out of range, then truncated to DATA_WIDTH bits.
- Serialization control: out_valid mirrors output_en directly. out_last is asserted combinatorially whenever row and col both equal ARRAY_SIZE-1 — held as a stable level while waiting on out_ready, not a pulse. output_done = out_valid && out_ready && out_last, firing the exact cycle the true final result transfers. row/col advance only on a genuine transfer (out_valid && out_ready), and reset to zero the cycle after output_done fires.

Handshaking with Neighboring Modules

Neighbor	Signals	Description
systolic_array	results (in)	Full accumulator grid, stable throughout the output phase
feeder (via accelerator_core)	output_en (in, = feeder.drain_done)	Serialization only proceeds once feeder has confirmed drain complete
feeder and systolic_array (via accelerator_core)	output_done (out)	Single pulse wired to both feeder.clear and systolic_array.clear
Host / accelerator_core boundary	out_ready (in); out_data, out_valid, out_last (out)	Standard AXI4-Stream output handshake

Section 5 — Dataflow and Timing

5.1 Vector-by-Vector Streaming

Each 8x8 pass streams one column of A and one row of B, ARRAY_SIZE words each, over the two independent lanes. A vector completes only once both lanes have delivered all ARRAY_SIZE of their words for that vector — words within a lane must arrive in order, but the two lanes may interleave, stall, or complete in either order relative to each other. The moment both lanes' current vector is complete, feeder presents one new wavefront (a full ARRAY_SIZE-wide column/row pair) to the skew buffer and pulses valid for exactly one cycle.

5.2 Diagonal Skew

Row 0 of the skew buffer is a direct wire — zero delay. Row i (for i from 1 to ARRAY_SIZE-1) has i chained register stages, each advancing on a valid pulse (not every clock cycle). This produces the diagonal wavefront pattern the array needs: at global pulse count p , PE(r,c) receives wavefront $k = p - r - c$, since row r 's own entry point receives wavefront k at pulse $(k+r)$ via feeder's skew, and that value then takes c further pulses to propagate horizontally across the array to reach column c .

5.3 Drain Phase

Once feeder observes the true final word on both lanes (a_last and b_last both latched), it self-triggers a drain sequence, continuing to pulse valid every cycle while shifting zeros through the skew buffer for DRAIN_LEN = $2 \times (\text{ARRAY_SIZE}-1)$ cycles.

```
Drain pulses 1 to (ARRAY_SIZE-1):
  Real data continues exiting the skew buffer -- row  $i$ 's own final
  real value clears the skew buffer at drain pulse  $i$ .

Drain pulses (ARRAY_SIZE) to  $2 \times (\text{ARRAY\_SIZE}-1)$ :
  All-zero wavefronts. The LAST real wavefront (which entered the
  skew buffer just before drain began) is still propagating
  horizontally/vertically across the array during this window,
  reaching the deepest PE, (ARRAY_SIZE-1, ARRAY_SIZE-1), exactly
  at the end of this phase.

One cycle after the last drain pulse: drain_done latches.
  This extra cycle gives the array's own registered accum_out
  values time to settle before output_processor begins reading them.
```

This drain length is the same delay derived from the standard systolic timing relationship: for the deepest PE to receive the last real wavefront, measured from the moment that wavefront first enters the skew buffer at row 0, requires exactly $2 \times (\text{ARRAY_SIZE}-1)$ cycles — matching both the skew buffer's own maximum depth and the array's own horizontal/vertical propagation depth simultaneously.

5.4 Output Serialization Timing

output_processor's out_valid mirrors output_en (wired to feeder's drain_done) directly, so serialization begins the same cycle drain_done latches. output_done is purely combinational — it becomes true the exact cycle row and col first both reach ARRAY_SIZE-1 while a genuine transfer is occurring (out_valid && out_ready), and self-resets row/col to zero on that same edge. Because output_done also clears feeder's own drain_done register on that identical edge, out_valid naturally returns to zero immediately afterward, with no separate latch needed to prevent re-firing.

5.5 Reset and clear

rst_n is the asynchronous, global reset, clearing every register in every module. clear is a single, synchronous, self-generated pulse — sourced by output_processor's own output_done — that resets feeder's staging/skew/counter state and every MAC unit's accumulator/pipeline registers on the same edge, preparing the whole chip for the next computation with no controller coordinating the sequence.

Section 6 — Pin Interface and Protocol

6.1 Physical Pin Budget

Slot A's total pin budget is 22, of which this design uses all 22:

Category	Count	Signals
Input-only	5	a_valid_i, a_last_i, b_valid_i, b_last_i, out_ready_i
Bidirectional	12	data[7:0] (shared) + a_ready_o, b_ready_o, out_valid_o, out_last_o
Analog (workaround)	1	Unused functionally; present to avoid a zero-width-vector synthesis quirk
Clock / reset	2	clk, rst_n
Power	2	1 DVDD + 1 DVSS

6.2 Physical Pin Assignment

Pin	Direction	Description
clk	in	System clock (25 MHz target)
rst_n	in	Active-low asynchronous reset
data[7:0]	bidir	Shared bus: activation/weight bytes in, result bytes out
a_valid	in	Activation byte on data is valid this cycle
a_ready	out	Chip can accept an activation byte
a_last	in	This is the true final activation byte of the stream
b_valid	in	Weight byte on data is valid this cycle
b_ready	out	Chip can accept a weight byte
b_last	in	This is the true final weight byte of the stream
out_valid	out	Result byte on data is valid this cycle
out_ready	in	Host can accept a result byte
out_last	out	This is the true final result byte of the pass

6.3 Host-Side Protocol Rules

Because a single physical bus carries all three logical channels, the host must observe two rules that would not be necessary with a dedicated bus per channel:

- Never assert a_valid and b_valid in the same cycle. Both share the one physical data bus, so at most one byte can genuinely be on the wire at a time. host_interface does not arbitrate or enforce this — it is a host implementation responsibility.
- Stop driving data the instant out_valid is observed high, and do not resume until the result phase ends. The chip's own direction over the shared bus is derived directly from its own out_valid (data_oe = out_valid), so the host must yield the bus at exactly that point to avoid electrical contention on the shared pin.

Neither rule applies to a wider-bus SoC integration using accelerator_core's native interface directly — there, a_data and b_data are independent buses and no sharing discipline is required.

6.4 Transfer Protocol

```
Activation transfer : a_valid = 1 AND a_ready = 1 -> one byte transferred
Weight transfer    : b_valid = 1 AND b_ready = 1 -> one byte transferred
Result transfer     : out_valid = 1 AND out_ready = 1 -> one byte transferred
```

Host protocol per computation:

1. Stream A and B bytes (never a_valid and b_valid together), asserting a_last / b_last on each lane's true final byte
2. Wait for out_valid = 1 (drain complete, results beginning)
3. Release the data bus; read 64 result bytes
4. Chip self-clears; ready for the next computation immediately

6.5 Comparison with AXI4-Stream

This Design's Signal	AXI4-Stream Equivalent	Notes
a_valid / b_valid	TVALID	Level, standard semantics
a_ready / b_ready	TREADY	Level, standard semantics
a_last / b_last	TLAST	Level, held stable through any stall, not a pulse
out_valid	TVALID (source)	Level, standard semantics
out_ready	TREADY (source)	Level, standard semantics
out_last	TLAST (source)	Level, standard semantics

Section 7 — Parameters

7.1 Design Parameters

Parameter	Default	Valid Values / Constraint	Used In
ARRAY_SIZE	8	Any power of 2	accelerator_core, feeder, systolic_array, mac_unit, output_processor
DATA_WIDTH	8	Any positive integer	accelerator_core, feeder, systolic_array, mac_unit, output_processor
ACCUM_WIDTH	21	$\geq$ enough bits for the target max K	accelerator_core, systolic_array, mac_unit, output_processor
SHIFT_BITS	0	0 = pure clip, no shift	accelerator_core, output_processor
SAT_MAX	$2^{(\text{DATA_WIDTH}-1)}-1$	Overridable for tighter calibration	accelerator_core, output_processor
SAT_MIN	$-2^{(\text{DATA_WIDTH}-1)}$	Overridable for tighter calibration	accelerator_core, output_processor

7.2 ACCUM_WIDTH Derivation

Maximum INT8 x INT8: $(-128) \times (-128) = 16,384 = 2^{14} \rightarrow 15$ bits unsigned

Maximum accumulated value for K=32 passes:

$$32 \times 16,384 = 524,288 = 2^{19}$$

2^{19} requires 20 bits unsigned (bit 19 must be set)

+ 1 sign bit = 21 bits signed

21-bit signed range: -1,048,576 to +1,048,575

Max value 524,288 $\leq$ 1,048,575 -- safely within range

ACCUM_WIDTH is a synthesis-time parameter with no RTL changes required to increase it; larger K (more accumulation passes) simply requires proportionally more accumulator bits, following the same derivation above.

Section 8 — AXI4-Stream Wrapper (Future SoC Reuse)

A dedicated, wide-bus AXI4-Stream wrapper is a planned but not yet implemented addition to this repository, reserved at `src/axi_wrapper/`. It is intended for SoC integration contexts that specifically require full AXI4-Stream protocol compliance (e.g., a shared system bus with existing AXI infrastructure) rather than `accelerator_core`'s native interface used directly.

Because `accelerator_core`'s native interface is already shaped as three independent AXI4-Stream-style channels (data/valid/ready/last per channel), the planned wrapper's job would be narrow: width conversion between a wider system TDATA bus and `accelerator_core`'s per-channel `DATA_WIDTH`, and straightforward signal renaming (TVALID/TREADY/TLAST mapping directly to this design's own valid/ready/last signals on each channel). `accelerator_core` itself requires no change to support this wrapper when it is built.

Section 9 — Testbench Plan

9.1 Simulation Toolchain

- Primary: cocotb (Python-based verification framework)
- Simulator: Icarus Verilog
- Python generates test vectors and computes expected results via NumPy; testbenches compare and report PASS/FAIL

9.2 Unit and Integration Testbenches

Testbench	Module	Key Checks
tb_mac_unit	mac_unit	Signed multiply, accumulation, clear priority, valid-gated A/B propagation
tb_systolic_array	systolic_array	Matrix multiply vs NumPy, PE-to-PE wiring (token-trace method), skew timing
tb_feeder	feeder	Stalls (per-lane and combined), lane imbalance, a_last/b_last edge cases, drain sequencing, clear/reset behavior, back-to-back runs, structural/parameterization checks, data value edge cases
tb_output_processor	output_processor	Saturation/quantization (including SHIFT_BITS and custom SAT bounds), backpressure, output_en behavior, output_done/out_last timing, reset behavior, data value edge cases
tb_accelerator_core	accelerator_core	Single-pass and multi-pass (k=2, k=4) matmul vs NumPy, saturation, back-to-back runs, output backpressure, identity-matrix sanity check, input stalls
chip_top_tb	chip_top (Slot A)	Reset/idle sanity check, one full matrix multiply through the real physical pad interface

9.3 Verification Methodology Notes

- Every testbench derives its sizing (ARRAY_SIZE, DATA_WIDTH, etc.) from the compiled DUT's own parameters rather than hardcoding values, so re-running with a different synthesis-time configuration exercises a genuinely different build with no test code changes.
- chip_top_tb specifically exercises the real GF180MCU I/O pad models (bidirectional and input-only cell types) rather than accelerator_core's plain digital ports, to catch pin-mapping and pad-timing issues that cannot appear at any lower level of the hierarchy.
- Test scope at the chip_top level is intentionally narrow: the pad-level protocol logic itself is already exhaustively covered at the accelerator_core level, so chip_top_tb's role is specifically to confirm chip_core/host_interface's physical pin wiring, not to re-verify behavior already proven correct one level down.